21CSE525T — AGENT TECHNOLOGY

Assignment 1.1 — Project Ideation, Environment Modeling & PEAS Matrix

Design Specification Document

1. Domain Definition

1.1 Domain

Education Technology (EdTech) intersecting Artificial Intelligence and Decision-Support Systems.

1.2 Agent Persona

The Placement Preparation Agent is an autonomous software agent that assists an engineering student through the campus-placement cycle. It continuously senses the student's coding, aptitude, core-subject and interview performance, reasons about the gap between the current state and the target company's requirements, and acts by generating, revising and enforcing a personalised preparation roadmap.

1.3 Problem Statement

Students preparing for campus placements face three compounding difficulties: (i) they cannot reliably identify which of their skills are weakest relative to a target company, (ii) they cannot allocate limited study hours optimally across coding, aptitude, core subjects and communication, and (iii) preparation plans go stale the moment the environment changes — a drive is announced early, a cut-off is revised, or a test score moves. Existing platforms deliver static, generic content and require the student to do the planning. The proposed agent closes this loop autonomously: it perceives change, re-deliberates, and re-plans without being asked.

1.4 Scope and Goals

- G1 — Maintain an accurate, continuously-revised model of the student's skill state.
- G2 — Maximise a defined placement-readiness utility subject to time and budget constraints.
- G3 — Re-plan autonomously when the environment changes (new drive, revised cut-off, new score).
- G4 — Coordinate with external agents (mentor agents, placement-cell agent) to secure resources.
- G5 — Reject unauthenticated or tampered instructions from untrusted hosts.

2. PEAS Profile

The PEAS framework specifies the task environment for a rational agent. The table below defines the complete PEAS profile for the Placement Preparation Agent.

Component	Specification for the Placement Preparation Agent
Performance Measure (P)	Weighted placement-readiness score U = w1·skill_gain + w2·placement_boost + w3·mentor_quality + w4·cost_efficiency + w5·time_efficiency, subject to Σwi = 1. Secondary measures: number of tier-1/2 companies for which the student becomes eligible, reduction in skill-gap count against the target company, mock-interview rating improvement, plan-adherence ratio, and mean time-to-replan after an environmental change.
Environment (E)	Student profile (CGPA, branch, backlog status); coding platforms (LeetCode, HackerRank) supplying solved-count and accuracy telemetry; aptitude testing engine; mock-interview mentors and their feedback; the campus placement cell publishing drive dates and eligibility cut-offs; a curated resource corpus (problem sheets, articles, question banks); the student's own calendar and

Component	Specification for the Placement Preparation Agent
	available study hours; and a monetary training budget.
Actuators (A)	Generate and publish a study plan; assign a graded problem ladder for a specific skill; schedule and book a mock-interview slot; issue reminder and nudge notifications; recommend learning resources; re-prioritise or drop an in-flight plan; dispatch signed negotiation messages (CFP, PROPOSE, ACCEPT, REJECT) to mentor agents; render the live CLI dashboard; and write revised beliefs back into the knowledge base.
Sensors (S)	Coding-platform score and delta feed; aptitude test-result feed; mock-interview rating and mentor feedback; topic-completion and time-on-task signals; placement-cell drive announcements and cut-off updates; company eligibility criteria; resume and portfolio signals (e.g. GitHub public repository count via REST API); ambient context (date, days-to-drive, weather affecting the study modality); and inbound network messages from peer agents.

3. Environment Characteristics

Classifying the task environment determines which agent architecture is appropriate. The classification below justifies the use of a deliberative BDI architecture layered over a reactive reflex core rather than a purely reflexive design.

Dimension	Classification	Justification
Observability	Partially Observable	The agent senses scores, submissions and announcements, but cannot observe the student's motivation, retention quality, the recruiter's private evaluation rubric, or the questions that will actually be asked. The agent must therefore maintain an internal belief state rather than act on raw percepts alone.
Determinism	Stochastic	The same preparation action does not yield a deterministic outcome. Interview results depend on panel subjectivity, competing candidates, and question sampling. Score improvements from a fixed study block vary.
Episodicity	Sequential	Every action changes the state on which subsequent actions operate. Mastering recursion is a prerequisite for dynamic programming; a poor mock interview alters the plan for the following week. There are no independent episodes.
Dynamism	Dynamic	The environment changes while the agent is deliberating: drives are advanced, cut-offs are revised, new companies register. The agent must re-deliberate mid-execution rather than compute a plan once.
Discreteness	Discrete	Percepts, actions and time are discretised. The agent selects from a finite catalogue of topics, resources, mentors and time slots; scores are integers in [0,100].
Agency	Multi-Agent (mixed cooperative / competitive)	The core preparation loop serves a single student, but the agent negotiates with self-interested Mentor Agents (competitive, opposing utility) and cooperates with the Placement Cell Agent (shared goal). Peer student agents introduce competition for limited drive slots.
Knowledge	Known	The transition and reward semantics are known to the

Dimension	Classification	Justification
		designer: prerequisite relations, eligibility rules and utility weights are explicitly encoded in the ontology, even though outcomes remain stochastic.

4. Autonomous Agent vs. Standard Object-Oriented Program

The following structural contrast establishes why this system is an agent rather than a conventional object-oriented application. Each row contrasts a defining agent property with the corresponding behaviour of an ordinary OO program.

Criterion	Autonomous Agent (this system)	Standard Object-Oriented Program
Thread of control	Owns its own thread of control. Daemon workers and the deliberation loop run continuously and initiate action independently.	Control is owned by the caller. An object's method executes only when it is invoked from outside.
Invocation model	Proactive — decides for itself when and whether to act, and may refuse or defer.	Reactive to invocation — a public method always executes when called; it cannot decline.
State semantics	Maintains beliefs: a revisable model of the world, updated from percepts, with revision history.	Maintains fields: passive data with no notion of evidence, staleness or revision.
Goal orientation	Goal-directed. Holds desires with dynamic priority and commits to intentions to achieve them.	Task-directed. Executes the specific function the programmer encoded; has no goal representation.
Adaptivity	Re-plans at runtime. A belief change can cause an in-flight intention to be dropped and replaced.	Follows a fixed control flow. Changing behaviour requires editing and recompiling source code.
Environment coupling	Situated and persistent. Continuously senses and acts on the environment across its whole lifetime.	Transient and decoupled. Reads inputs, computes, returns, and terminates.
Interaction model	Social. Communicates with other agents via a speech-act protocol (CFP / PROPOSE / ACCEPT / REJECT) and negotiates outcomes.	Mechanical. Calls methods on other objects; no negotiation, no possibility of refusal or counter-offer.
Failure handling	Deliberative recovery — reconsiders desires and selects an alternative plan when an action fails.	Exception propagation — throws upward and relies on the caller to decide what happens next.
Autonomy over data	Encapsulates behaviour and control. Decides whether to honour an incoming request, and authenticates its source.	Encapsulates data only. Any caller holding a reference may invoke any public method.

5. Agent Architecture Overview

The agent is implemented as a layered hybrid architecture. A reactive layer guarantees fast responses to urgent percepts, while a deliberative BDI layer performs goal reasoning over a longer horizon.

Layer	Unit	Responsibility
Security Gateway	5	Verifies HMAC-SHA256 signatures, host identity, freshness and role authorisation on every inbound message before it reaches the agent core.
Reactive Layer	1	Condition-action rule engine. Fires immediate responses to urgent percepts (imminent drive, collapsed score, idleness).
Perception & Concurrency	2	Daemon worker threads poll external sources and publish signed percepts onto a thread-safe event bus; socket endpoints exchange serialised state with remote agents.
Social Layer	3	Contract-Net negotiation with mentor agents and constrained utility maximisation over the resource catalogue.
Deliberative Core	4	Belief base, desire set with dynamic priority, and an intention stack supporting mid-execution re-planning; backed by the domain ontology and live REST feeds.
Presentation	5.2	Live CLI dashboard rendering skill bars, active desires, current intention and security counters.

6. Deliverable Mapping

Rubric Component	Marks	Where satisfied in this document
Domain & Persona Clarity	1	Section 1 — domain, persona, problem statement, scope and goals.
PEAS Profile Completeness	2	Section 2 — full PEAS table; Section 3 — environment characteristics classification.
Agent vs. Object Structural Contrast	2	Section 4 — nine-criterion structural contrast table.